

Experiment 5

Digital Input/Output Interfacing and Programming

Objective

The objective of this lab is to give you a hands-on exposure to the programming of I/O, which when executed by the microcontroller (TI TM4C123, an ARM Cortex-M4 based microcontroller) simply blinks LED on the development board.

Configuring Peripherals

The fundamental initialization steps required to utilize any of the peripheral are:

1. Enable clock to the peripheral
2. Configure pins required by the peripheral
3. Configure peripheral hardware/functionality

Introduction to GPIO

A microcontroller communicates with the outside world either by setting the voltage on the pin high (usually 5V) or low (usually 0V) or reading the voltage level of an input pin as being high (1) or low (0). We refer to these pins as general purpose input output (GPIO) pins. Any GPIO pin can be configured through software to be either a digital input or a digital output. GPIO outputs let you translate logical values within your program to voltage values on output pins and voltage outputs help your microcontroller exert control over the system in which it is embedded.

Where is LED?

The TivaC LaunchPad comes with an RGB LED and two switches. *Switches are connected at one end to the GPIOs and other to ground.* These switches and LED can be configured for any custom application. Table 5.1 provides the Launchpad pins for LEDs. Figure 5.1 shows the physical connection of the onboard LED.

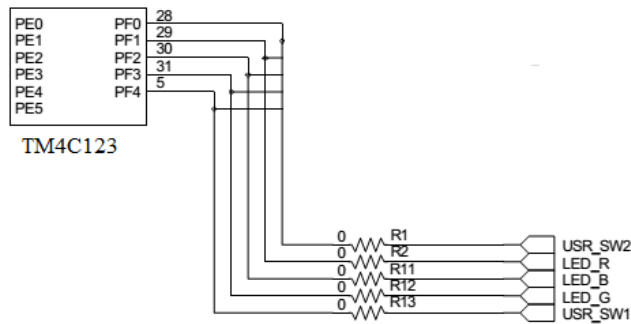


Figure 5.1: LED Schematic

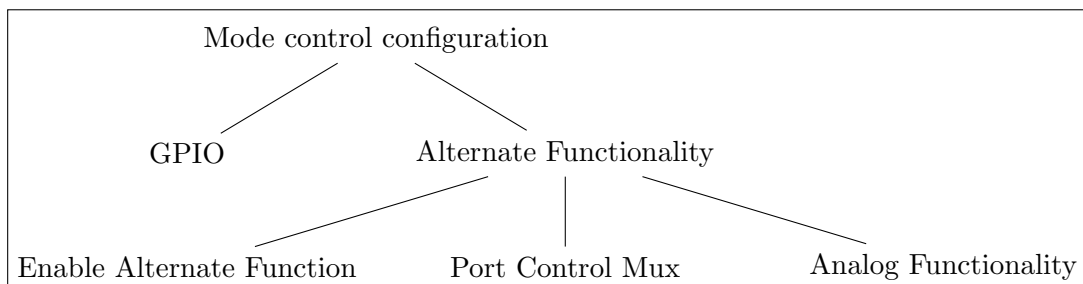
GPIO pin	Pin Function	USB Device
PF1	GPIO	RGB LED (Red)
PF2	GPIO	RGB LED (Blue)
PF3	GPIO	RGB LED (Green)

Table 5.1: RGB LED Signals

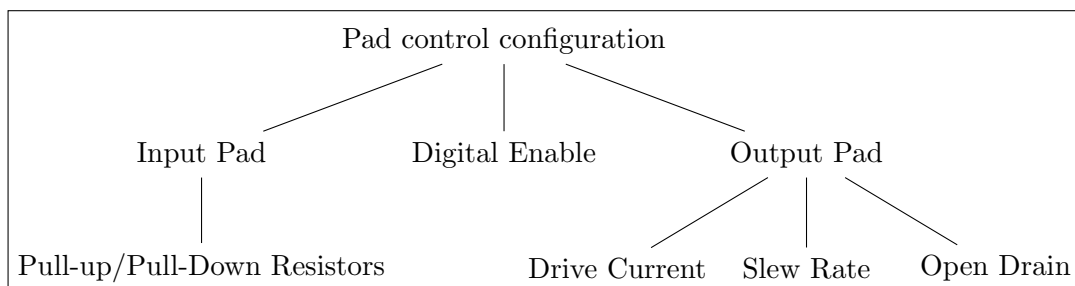
Configuring GPIOs

The basic steps required to configure GPIOs are:

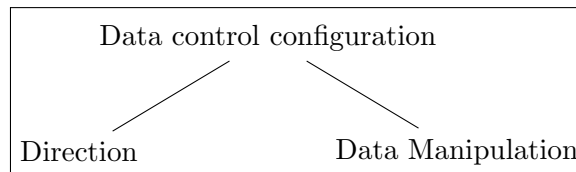
1. Clock configuration
2. Mode control configuration



3. Pad control configuration



4. Data control configuration



Note: If GPIO pins are locked (PF0 and PD7), then they need to be unlocked using GPIO Lock and GPIO Commit registers right after enabling the clock.

Structure of the Program

The overall structure of this program is illustrated below. The program begins by including the addresses relevant peripheral registers. Main routine follows the initialization steps described above and then enters an infinite loop which toggles an LED and waits for sometime.

```

#define register_name    (*(volatile unsigned long *) register_address)

int main(void) {
    // Enable peripherals
    ... (1) ...
    // Configure pins
    ... (2) ...
    while(1) {
        // Turn ON LED
        ... (3) ...
        // Delay for a bit
        ... (4) ...
        // Turn OFF LED
        ... (5) ...
    }
}

```

Pseudo code to blink LED

LED Configuration

We will follow the steps stated above to configure the on-board LED.

Enabling the Clock

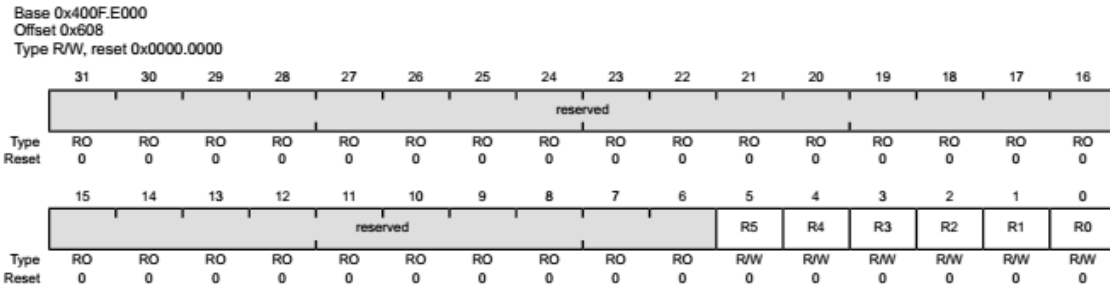
The RCGCGPIO register provides software the capability to enable and disable GPIO modules in Run mode. When enabled, a module is provided a clock and access to module registers. When disabled, the clock is disabled to save power and accessing a module register generates a bus fault. This register is shown in Figure 5.2. The clock can be enabled for the GPIO port F by asserting the 6th bit of RCGCGPIO register.

Following command can be used to enable clock signal for GPIO port F

```

SYSCTL_RCGCGPIO_R |= 0x20;    // (1)

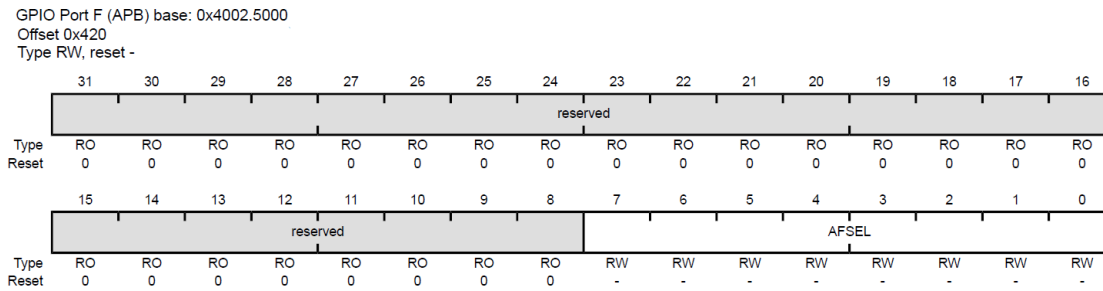
```



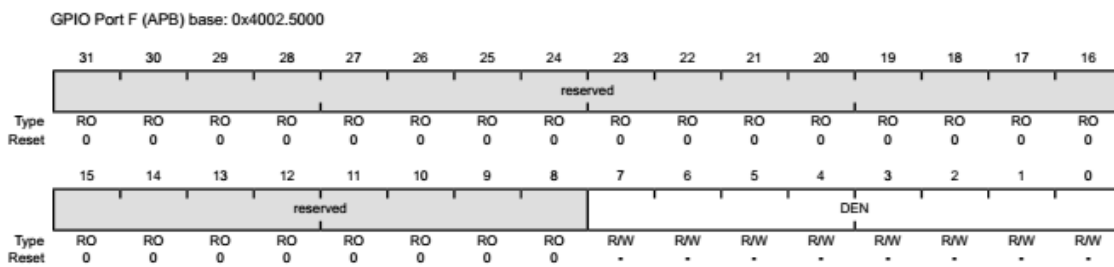
General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Configuring the Pin as Output

After enabling the clock, it is necessary to configure the required pins. In this case, a single pin (PF3) must be configured as an output. To use the pin as a digital input or output, alternate functionality is turned off for the corresponding bit in the GPIOAFSEL (Figure 5.3), digital circuitry is turned on in GPIODEN register (Figure 5.4) and then setting a bit in the GPIODIR (Figure 5.5) register configures the corresponding pin to be an general purpose output pin.



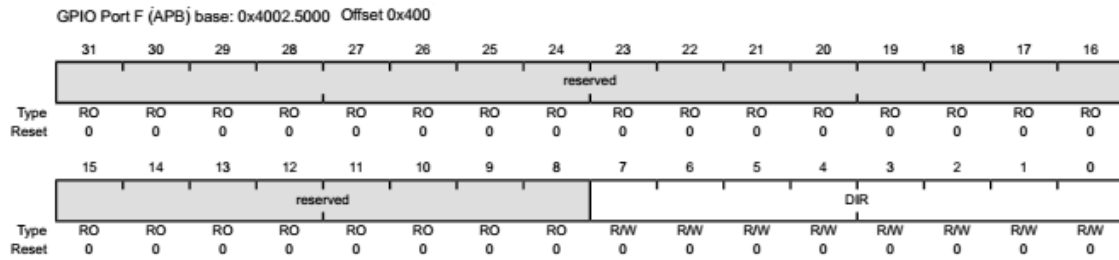
GPIO Alternate Functionality (GPIOAFSEL)



GPIO Digital Enable (GPIODEN)

The commands used to set the corresponding bits in GPIODEN and GPIODIR registers are given as follows

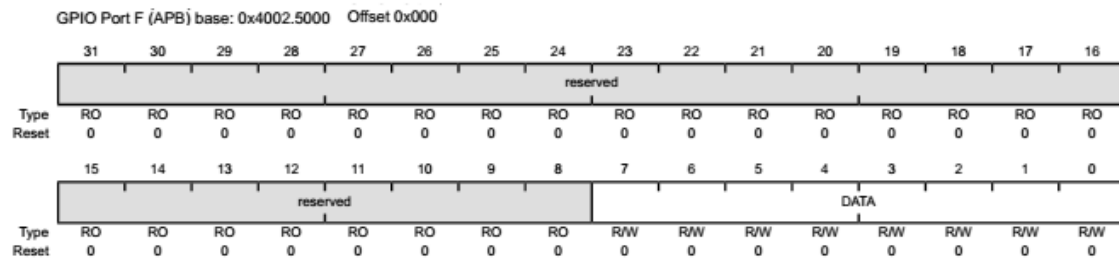
```
GPIO_PORTF_AFSEL_R &= ~GPIO_PORTF_PIN3_EN;
GPIO_PORTF_DEN_R |= 0x08;
GPIO_PORTF_DIR_R |= 0x08;      // (2)
```



GPIO Direction (GPIODIR)

Toggle the LED

After configuring the LED (pin PF3) as an output, now we want to toggle it after regular intervals. LED can be turned ON and OFF by setting and resetting the corresponding bits in the GPIODATA register.



GPIO Data (GPIODATA)

The commands for toggling LED are as follows

```
GPIO_PORTF_DATA_R |= 0x08;           // (3)
GPIO_PORTF_DATA_R &= (~0x08);       // (5)
```

Introducing a Delay

We cannot observe the toggling of the LED because of very high frequency. So, we introduce a delay loop in order to observe the toggle sequence of the LED. The syntax for the loop is shown in the following

```
int counter = 0;
while(counter < 200000){ // (4)
    ++counter;
}
```

Source Code

The complete source code for the program is given as follows

Example 5.1.

```
#define SYSCTL_RCGCGPIO_R      (*((volatile unsigned long *)0x400FE608))

#define GPIO_PORTF_AFSEL_R     (*((volatile unsigned long *)0x40025420))
#define GPIO_PORTF_DEN_R       (*((volatile unsigned long *)0x4002551C))
#define GPIO_PORTF_DIR_R       (*((volatile unsigned long *)0x40025400))
#define GPIO_PORTF_DATA_R      (*((volatile unsigned long *)0x400253FC))

#define GPIO_PORTF_CLK_EN      0x20
#define GPIO_PORTF_PIN3_EN     0x08
#define LED_ON                  0x08
#define LED_OFF                 ~(0x08)
#define DELAY                   200000

int main(void)
{
    volatile unsigned long ulLoop;

    /* Enable the GPIO port that is used for the on-board LED. */
    SYSCTL_RCGCGPIO_R |= GPIO_PORTF_CLK_EN;

    /* Do a dummy read to insert a few cycles after enabling the
       peripheral. */
    ulLoop = SYSCTL_RCGCGPIO_R;

    /* Enable the GPIO pin for the LED (PF3). Enable the GPIO pin for
       digital function and set the direction as output. */
    GPIO_PORTF_AFSEL_R &= ~GPIO_PORTF_PIN3_EN;
    GPIO_PORTF_DEN_R   |= GPIO_PORTF_PIN3_EN;
    GPIO_PORTF_DIR_R   |= GPIO_PORTF_PIN3_EN;

    // Loop forever.
    while(1)
    {
        // Turn on the LED.
        GPIO_PORTF_DATA_R |= LED_ON;

        // Delay for a bit.
        for(ulLoop = 0; ulLoop < DELAY; ulLoop++){

        // Turn off the LED.
        GPIO_PORTF_DATA_R &= LED_OFF;

        // Delay for a bit.
        for(ulLoop = 0; ulLoop < DELAY; ulLoop++){

    }
}
```

Exercises

Repeat the above procedure for RED and BLUE LED.